

Parallel A* Password Recovery

Sequential, Pthreads, OpenMP, OpenCilk, CUDA and Hybrid CPU+GPU
Implementations

Parallel & Distributed Systems (Homework 4)
Apostolos Pantazis (10910)

September 2026

Abstract

This report presents an MD5 password-recovery tool formulated as an **A* state-space search** over a prefix tree and parallelized across five paradigms: POSIX threads, OpenMP, OpenCilk, CUDA, and a Hybrid Pthreads + CUDA design. The search uses a uniform cost model over a fixed 76-symbol alphabet with a bounded beam to cap memory. We describe the algorithmic formulation, the parallelization strategy of each implementation, and an experimental evaluation on a 28-core host equipped with an NVIDIA GeForce RTX 4070 Ti SUPER. All implementations recover length-4 targets correctly; the Hybrid design additionally succeeds on the harder length-5 space where the CPU-only variants exhaust their search budget.

1 Problem Formulation

Given the MD5 digest of an unknown password of known length L , the goal is to recover the plaintext. Because cryptographic hashes exhibit the *avalanche effect*, there is no continuous notion of “distance” between a partial candidate’s hash and the target. We therefore do not use the hash to guide the search; instead we treat the problem as a graph search over a prefix tree and use A* as a systematic, memory-bounded traversal.

State-space model.

- **State:** a candidate prefix string s at depth d .
- **Alphabet:** a fixed 76-symbol set — lowercase, uppercase, digits, and the symbols `!@#$%^&*()-_+=.`
- **Path cost:** $g(n) = d \cdot \text{EDGE_COST}$, with $\text{EDGE_COST} = 1.0$.
- **Heuristic:** $h(n) = (L - d) \cdot \text{EDGE_COST}$ (remaining depth; admissible).
- **Evaluation:** $f(n) = g(n) + h(n) = L$, uniform across all candidates of the target length.
- **Goal test:** MD5 verification is performed *only* when a candidate reaches depth L (a leaf).

Since f is uniform, A* here behaves as a systematic frontier expansion. To bound memory we apply **beam trimming**: when the priority queue exceeds $\text{BEAM_CAP} = 3\,000\,000$ states, a `quickselect` keeps the best $\text{BEAM_KEEP} = 1\,200\,000$ and re-heapifies. There is **no brute-force fallback and no training** of any kind: a case reported as *not found* is a genuine failure within the search’s own node/time budget.

2 Software Architecture

The project shares a common core across all targets:

File	Role
<code>astar_heap.{h,c}</code>	Min-heap priority queue + beam-trim (quickselect + heapify).
<code>md5.{h,c}</code>	Standalone RFC 1321 MD5 for CPU targets.
<code>crack_astar.c</code>	Sequential baseline.
<code>crack_astar_pthreads.c</code>	POSIX threads, atomic dynamic task queue.
<code>crack_astar_omp.c</code>	OpenMP dynamic loop scheduling.
<code>crack_astar_opencilk.c</code>	OpenCilk <code>cilk_for</code> work-stealing.
<code>crack_astar_cuda.cu</code>	CPU-driven heap + GPU batch leaf hashing.
<code>crack_astar_hybrid.cu</code>	Pthreads + per-thread CUDA streams.

Priority queue. A binary min-heap ordered by f supports `push` and `pop` in $O(\log n)$. Beam trimming uses `quickselect` to partition the best k states in expected $O(n)$ before re-heapifying, avoiding a full $O(n \log n)$ sort.

3 Parallelization Strategies

3.1 Pthreads

The root search space is partitioned into two-character prefix tasks ($76^2 = 5776$ independent sub-searches for $L \geq 2$). Worker threads claim tasks dynamically through an atomic counter (`atomic_fetch_add`), which provides natural load balancing. Each thread owns a *private* priority queue, eliminating lock contention on the heap. An atomic flag signals early termination the moment any thread finds the match.

3.2 OpenMP

Uses the same 5776-task decomposition, distributed with `#pragma omp parallel for schedule(dynamic, 1)`. A per-thread queue is preallocated and reused (indexed by `omp_get_thread_num()`) to avoid repeated heap allocation.

3.3 OpenCilk

Exposes the sub-tasks to the work-stealing runtime with `cilk_for`; each iteration runs an independent A* traversal with its own local heap. Worker count is controlled via `CILK_NWORKERS` or `-threads`.

3.4 CUDA

The CPU performs the pointer-based traversal and heap operations while completed leaf candidates accumulate in a batch buffer. When the batch fills (or a node threshold is reached) the candidates are copied to the GPU and hashed in parallel by an optimized MD5 kernel. Here `-threads` denotes the GPU *batch size*, not a CPU thread count.

3.5 Hybrid Pthreads + CUDA

Combines CPU dynamic task distribution with GPU batch execution. Each POSIX worker is assigned (i) its own CUDA stream (`cudaStreamCreate`) for concurrent GPU execution, (ii) pinned host memory (`cudaHostAlloc`) for DMA transfers, and (iii) private device buffers/flags to avoid device-side synchronization.

4 Build & Run

```

make all                                     # sequential, pthreads, openmp
make crack_astar_opencilk CILK_CC=/opt/opencilk/bin/clang
make crack_astar_cuda NVCC_ARCH=sm_89      # match GPU compute capability
make crack_astar_hybrid NVCC_ARCH=sm_89

./crack_astar 4 PtYg
./crack_astar_pthreads 4 PtYg --threads 8
./crack_astar_hybrid 5 yJuq\ ) --threads 16 --batch-size 262144

```

5 Experimental Setup

CPU	28 logical cores (WSL2, Linux 6.6)
GPU	NVIDIA GeForce RTX 4070 Ti SUPER (sm_89)
Compilers	gcc -O2 -std=c11; OpenCilk clang; nvcc -O3
Alphabet	76 symbols; EDGE_COST=1.0
Beam	BEAM_CAP=3M, BEAM_KEEP=1.2M
Test set	six random length-4 targets; four length-5 (exploratory)

Timings are wall-clock seconds for the A* phase. Because thread task order affects which leaf is reached first, node counts vary slightly across thread counts — this is inherent to a nondeterministic parallel frontier and does not affect correctness.

6 Results

6.1 CPU scaling on length-4 targets

Table 1 lists representative wall-clock times. Sequential timings vary strongly with the target because A* frontier ordering reaches some targets much sooner than others. The Pthreads and OpenMP variants scale near-linearly with thread count; OpenCilk is fastest at low thread counts due to its lightweight work-stealing, but its timing is noisier because stealing changes which leaves are explored first.

Table 1: Wall-clock time (s) on length-4 targets. *seq* is single-threaded; parallel columns use the stated thread count. CUDA uses batch size 262144.

Target	seq	pth-8	pth-32	omp-8	omp-32	cilk-8	cuda
PtYg	1.593	0.233	0.085	0.524	0.157	0.146	0.126
j&mU	2.987	0.058	0.022	0.122	0.041	0.035	0.404
=h#B	0.176	0.422	0.154	0.943	0.298	0.211	0.818
el31	3.375	0.024	0.010	0.052	0.018	0.084	1.021
iEl(	3.181	0.049	0.020	0.107	0.040	0.170	1.175
2h-p	1.084	0.301	0.111	0.684	0.211	0.097	1.255

6.2 Pthreads speedup

Table 2 reports the Pthreads speedup over its own single-thread run for the PtYg target. Scaling is close to ideal up to 8 threads and continues improving to 32, reflecting the coarse-grained, contention-free task decomposition.

Table 2: Pthreads self-relative speedup for target PtYg (baseline: pthreads with 1 thread, 1.863 s).

Threads	Time (s)	Speedup	Efficiency
1	1.863	1.00×	100%
2	0.928	2.01×	100%
4	0.457	4.07×	102%
8	0.233	8.01×	100%
16	0.133	14.05×	88%
32	0.085	21.88×	68%

6.3 Hybrid: length-4 sweeps

On length-4 targets the Hybrid design is competitive with the best CPU variants, with a sweet spot around 4–8 threads and batch size 262 144 (Table 3). Beyond that, GPU launch overhead and increased frontier exploration erode the benefit.

Table 3: Hybrid thread sweep, batch size 65 536, target PtYg (length 4).

Threads	Time (s)	Nodes	GPU batches
1	0.234	242 005	277
2	0.123	241 799	276
4	0.078	243 983	279
8	0.063	249 377	283
16	0.064	258 052	289
32	0.107	444 744	521

6.4 Hybrid: length-5 (headline finding)

Length-5 is the key result. Every CPU-only implementation exhausts its search budget at length 5 (100% failure), whereas the Hybrid CPU+GPU design **recovers all four exploratory targets**. Table 4 shows the batch-size sweep at 16 threads: larger batches amortize GPU launch overhead and cut both time and total kernel launches sharply.

Table 4: Hybrid batch-size sweep, 16 threads, length-5 targets. All *found*.

Target	Batch	Time (s)	Nodes	GPU batches
yJuq)	16 384	3.315	17 026 048	77 935
	65 536	1.293	11 670 923	13 350
	262 144	0.953	11 012 055	3 143
	1 048 576	0.973	11 019 354	780
ntG0y	16 384	1.764	8 776 386	40 175
	65 536	0.833	7 333 648	8 386
	262 144	0.541	6 040 837	1 721
	1 048 576	0.594	6 038 149	424
(K5cq	16 384	6.664	31 718 287	145 193
	65 536	3.753	31 474 060	36 011
	262 144	2.692	31 416 671	8 980
	1 048 576	2.581	31 481 708	2 245

7 Discussion

- **Decomposition quality.** The 5 776 two-character prefix tasks give plenty of parallel slack for a 28-core host, and per-thread private heaps remove the main contention point, which is why Pthreads scales near-ideally to 8 threads.
- **Runtime differences.** OpenMP is consistently a bit slower than Pthreads at the same thread count due to scheduling overhead. OpenCilk wins at low thread counts but its work-stealing makes the number of expanded nodes — and therefore time — less predictable.
- **GPU trade-off.** Pure CUDA rarely beats a well-parallelized CPU at length 4 because the bottleneck is CPU-side frontier expansion, not hashing. The GPU only pays off when the leaf-verification volume is large, which is exactly the length-5 regime.
- **Batch size matters.** For the Hybrid design, small batches (16 384) drown in kernel-launch overhead; batches of 262 144–1 048 576 minimize launches and give the best time.
- **Oversubscription.** At 32 threads several Hybrid cases show a jump in node count and time — contention for the single GPU plus frontier divergence outweighs added CPU parallelism.

8 Verification

Automated functional tests (`run_verification_tests.sh`) confirm every built implementation recovers the reference target `PtYg`. All six binaries compile cleanly under `-Wall -Wextra` with no warnings and produce the correct plaintext.

Implementation	Result
Sequential	PASS — recovered <code>PtYg</code>
Pthreads	PASS — recovered <code>PtYg</code>
OpenMP	PASS — recovered <code>PtYg</code>
OpenCilk	PASS — recovered <code>PtYg</code>
CUDA	PASS — recovered <code>PtYg</code>
Hybrid	PASS — recovered <code>PtYg</code>

9 Conclusion

All six implementations are correct and reproducible. On length-4 targets the CPU parallel variants achieve near-ideal speedups (Pthreads reaches $\approx 22\times$ at 32 threads), while pure GPU offloading is limited by CPU-side frontier expansion. The decisive result is at length 5: only the Hybrid Pthreads + CUDA design, combining dynamic CPU task distribution with per-thread CUDA streams and large GPU batches, extends the reach of the search into a space where every CPU-only implementation fails. This confirms that heterogeneous CPU+GPU cooperation — not GPU offloading alone — is what unlocks the harder search regime.

You can find all the necessary files for this project on this GitHub repository:

<https://github.com/pantazisa/Homework>